
Markov Decision Process

Dillon Adli Gunawan Chi Hang Po
Department of Computer Science
Faculty of Engineering
The Chinese Univeristy of Hong Kong

Abstract

This paper explores the application of Markov Decision Processes (MDPs) for use in problems involving decision-making under uncertainty. We examine two basic problems, the Gambler's Problem and the Gridworld environment, demonstrating how MDPs provide a mathematical framework for optimal decision-making, through the providing an optimal policies. We implement two different algorithms, Value and Policy Iteration, to solve these MDPs, comparing their convergence properties and solution quality. Our results show that both algorithms successfully converge to optimal policies, with Value Iteration requiring more iterations but fewer computational resources per iteration, but Policy Iteration overall being faster. The paper serves as a demonstration of MDP fundamentals and their practical implementation and solving using Python, and a comparison of different solving algorithms, them being Value and Policy Iteration.

1 Introduction

Decision-making under uncertainty is a common feature in modeling systems for real-world applications, ranging from robotics to finance. To handles this, Markov Decision Processes (MDPs) provide a mathematical framework for modeling decision-making in situations where outcomes are partly random and partly under the control of a decision maker, through a structured approach to these problems by formalizing the concepts of states, actions, transitions, and rewards [1]. The framework enables the solving of these situations, by finding optimal policies that maximize expected cumulative rewards over time. Ever since their introduction, MDPs have been fundamental to reinforcement learning, and artificial intelligence.

This report examines two basic problems that illustrate the core concepts of MDPs: the Gambler's Problem [2] and Gridworld [3]. The Gambler's Problem represents a simple stochastic game where optimal betting strategies must balance risk and reward, similar to the much more complex real-life stock market exchange. Gridworld provides a path-finding task where an agent must learn optimal movement policies, similar to path-finding problems of autonomous driving and robotic systems. Both problems, while simple, capture the essential characteristics of more complex real-world decision problems.

The primary goals of this report are to:

1. An explanation of the features of an MDP, and Value and Policy Iteration algorithms
2. An implementation of MDPs for the Gridworld and Gambler's Problem, and Value and Policy Iteration algorithms to solve them using Python
3. A comparison of the two solving algorithms, their convergence properties and their solution quality

2 Background

2.1 Definition of Markov Decision Process

Markov Decision Process, as the name would imply, depends on the Markov Property, which means that the possible future states and effect of actions only depend on the current state. The Markov Property can be written as [1]:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t) \quad (1)$$

If the Markov Property is applicable, a problem can be defined as an MDP, by defining, the states, actions, transitions, rewards, policy and discount factor [1]:

- **States:** S representing the set of all possible states, with $s \in S$
- **Actions:** $A(s)$ representing the set of all possible action in a given state s , with $a \in A(s)$
- **Rewards:** $r = R(s, a)$, r representing the reward taking an action a in a state s
- **Transition Model:** Will describe how an action can affect the next possible states, hence can be defined as $P(s'|s, a)$, representing the probability of reaching the next state s' by taking an action a at a state s
- **Policy:** $\pi(s) = a$, representing the action a that is output and will be taken with a given policy π , which tells what an agent will do in a current state s . (Note that it can also be defined as a probability distribution across all possible actions, but for this report we consider it to be deterministic)
- **Discount Factor (γ):** The factor which future rewards are valued by (Note that $\gamma \leq 1$)

2.2 Optimal Policy and Value functions

When modeling an MDP, desired outcomes/states are given positive rewards, while undesired outcomes and states are given negative rewards, and reaching the most desirable outcome. If a policy (π) is a mapping of all possible current states to an action, then the optimal policy (π^*) will find the most optimal action at any state to maximize rewards. Hence, the goal of modeling a problem as an MDP is to use the MDP to find π^* .

Hence, algorithms have been made to find the optimal policy at a given MDP, such as Value and Policy Iteration. The core concepts behind these algorithms is the Value functions, $V^\pi(s)$, and Q-function, $Q^\pi(s, a)$ [1].

$V^\pi(s)$ represents the expected reward from starting at a state s and following a policy π , the sum of the immediate reward of taking an action, $\pi(s) = a$, at a state s , and the expected future rewards. The expected future rewards can be recursively defined as summation of product of the value function of the next state, s' , for all s' and the probability of reaching s' given s and a , multiplied by γ (Note that the summation in the equation is equivalent to the expected value function) [1]:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P(s'|s, \pi(s)) V^\pi(s') \quad (2)$$

$Q^\pi(s, a)$, represents the expected reward of taking an action a at state s , which is defined by addition of the immediate reward and the expected future rewards following a policy π [1]:

$$Q^\pi(s, a) = R(s, a) + \gamma \mathbb{E}[V^\pi(s')] \quad (3)$$

3 Methodology and Implementation

3.1 Gambler's Problem

The Gambler's Problem is defined as an agent, which has a current capital, x , and has a goal amount to reach, N . The agent can gain capital by betting any integer amount from 0 up to $\min(N - x, x)$, to prevent reaching more than the target amount. With probability p , the gambler wins the bet and gains the bet amount; with probability $1 - p$, the gambler loses the bet. By defining this problem as an MDP and solving for the optimal policy, you can find the optimal bet to reach N , at any current capital/state [2].

- **States:** All possible states can be defined as all possible amounts for the current capital of an agent, hence can be defined as the set $\{n | 0 \leq n \leq N\}$
- **Actions:** By the problem definition, at any current capital x , we can define the possible bet as the set $\{n | 0 \leq n \leq \min(N - x, x)\}$
- **Rewards:** We want to incentivize reaching the N , so we set $R(N, a) = 1$, and otherwise 0.
- **Transition Model:** From any current capital x , if we bet an amount n , we can only reach either state $x + n$ or $x - n$ by winning or losing the bet respectively. Therefore, we can set the $P(x + n | x, n) = p$, and $P(x - n | x, n) = 1 - p$, and otherwise 0

3.2 Grid Problem

The Grid World Problem models an agent navigating a 2D grid to reach a goal state while avoiding negative terminal (bad) states and wall cells. At each step the agent chooses a movement direction, but the environment is stochastic: with some probability the agent slips into a perpendicular direction instead of moving as intended [3]. By defining this environment as an MDP and solving for the optimal policy, one obtains an optimal action for every non-terminal cell:

- **States:** Each valid (non-wall) grid cell is a state, i.e.

$$\mathcal{S} = \{(i, j) \mid 0 \leq i < \text{rows}, 0 \leq j < \text{cols}, (i, j) \notin \mathcal{W}\},$$

where $\mathcal{W}$ is the set of blocked (wall) cells.

- **Actions:** The action set is the four movement directions

$$\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}.$$

If an action would move the agent off-grid or into a wall, the agent remains in the same state.

- **Rewards:** Let $\mathcal{G}$ be the set of goal states and $\mathcal{B}$ the set of bad (obstacle) terminal states. In this Grid World, goal states have reward +1, bad terminal states have reward -1, wall cells have reward 0 (and cannot be entered), and all other states have a living cost reward of -0.04.
- **Transition Model:** Terminal states are absorbing: if $s \in \mathcal{G} \cup \mathcal{B}$ then $P(s \mid s, a) = 1$ and $P(s' \mid s, a) = 0$ for $s' \neq s$. For non-terminal s , let s_{int} be the next state produced by the intended action, and let $s_{\perp 1}, s_{\perp 2}$ be the next states produced by the two perpendicular actions. With noise parameter α (set to 0.2), the transition probabilities are:

$$P(s' \mid s, a) = \begin{cases} 1 - \alpha, & s' = s_{\text{int}}, \\ \alpha/2, & s' \in \{s_{\perp 1}, s_{\perp 2}\}, \\ 0, & \text{otherwise.} \end{cases}$$

3.3 Value and Policy Iteration

To solve for optimal policy for these MDPs, we will use Value and Policy iteration. The core of Value Iteration is finding the optimal value function ($V^*(s)$) for an MDP. The optimal value function is not based of a policy, but is the maximum possible value function starting from a state s and is calculated iteratively. To calculate the more optimal value function at a state s , we maximize the sum of immediate reward and the expected future rewards, by varying a [4]. In each iteration, the algorithm will sequentially go through perform the calculation of the optimization of their value function once. The calculation for each state can be defined as[4]:

$$V_{k+1}(s) = \max_{a \in A(s)} [R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V_k(s')] = \max_{a \in A(s)} [Q(s, a)] \quad (4)$$

After finding the optimal value function, you can find the optimal policy, by selecting which action at a state, will maximize immediate and future rewards[4]:

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} [R(s, a) + \gamma \sum_{s' \in S} P(s' | s, a) V^*(s')] \quad (5)$$

Instead of finding the optimal value function, Policy iteration directly improves the policy each iteration. Policy Iteration is based on policy evaluation and policy improvement [5]. Policy evaluation

is finding the value function (V^π) for a certain policy iteratively, similar to value iteration, but instead of selecting the action that maximizes the value function, we select the action based on the current policy [5]:

$$V_{k+1}^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P(s'|s, \pi(s)) V_k^\pi(s') \quad (6)$$

We then use the value function from policy evaluation, to do policy improvement. We do this by selecting the action that maximizes immediate reward and future expected reward based on the value function from policy evaluation[5]:

$$\pi_{k+1}(s) = \operatorname{argmax}_{a \in A} [R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V^\pi(s')] \quad (7)$$

Hence, by setting initial value functions and policy, for value and policy iteration respectively, we can define their algorithms as:

```

V(s) = 0 for each state s in S
repeat
  for each state s in S
    for each action a in A
      Q(s,a) = R(s,a) +  $\gamma \sum_{s'} P(s'|s,a) V(s')$ 
    V(s) = max_a Q(s,a)
until V converges

```

(a) Value Iteration Algorithm

```

Choose  $\pi_0$  and set i=0
repeat
  for each state s in S
    Calculate  $V^{\pi_i}(s)$ 
  for each state s in S
    Compute  $\pi_{i+1}(s)$ 
  i = i+1
until  $\pi$  converges

```

(b) Policy Iteration Algorithm

3.4 Implementation Details

Both problems and algorithms, are implemented in Python (Source code can be found here: https://github.com/dev-stevenp/estr2018_report) with these parameters:

- For Gambler's Problem: $p = 0.4$, $N = 100$, and $\gamma = 1$ and 0.9 to compare solution and convergence properties.
- For GridWorld: $p = 0.8$, $\gamma = 0.9$ or 0.99, comparing optimal policy under different discount factors and algorithms.

4 Experiments and Results

4.1 Gambler's Problem

Experiment Parameters	Time Taken (s)	No. of Iterations
Value Iteration, $\gamma = 1$	5.56	63
Policy Iteration, $\gamma = 1$	4.54	9
Value Iteration, $\gamma = 0.9$	4.49	53
Policy Iteration, $\gamma = 0.9$	2.14	9

Table 1: Comparing time efficiencies of different algorithms in the Gambler's Problem

One key observation to make is the difference varying γ makes. At $\gamma < 1$, expected future rewards are not valued as much, so the optimal policy tries to minimize the number of bets made, to reduce its reliance on future rewards. Hence, the optimal policy will always bet the amount needed to reach N whenever it can. However, at $\gamma = 1$, the optimal policy is more likely to do smaller bets to slowly reach N , as even if they lose the small bet, they still have enough current capital to reach N in one bet.

When comparing Value and Policy Iteration, It can be seen they come to the same solution, when the parameters are similar, but their main difference lies in the time taken and number of iterations. Though Value iteration uses less time per iteration, the number of iteration makes it overall slower than policy iteration.

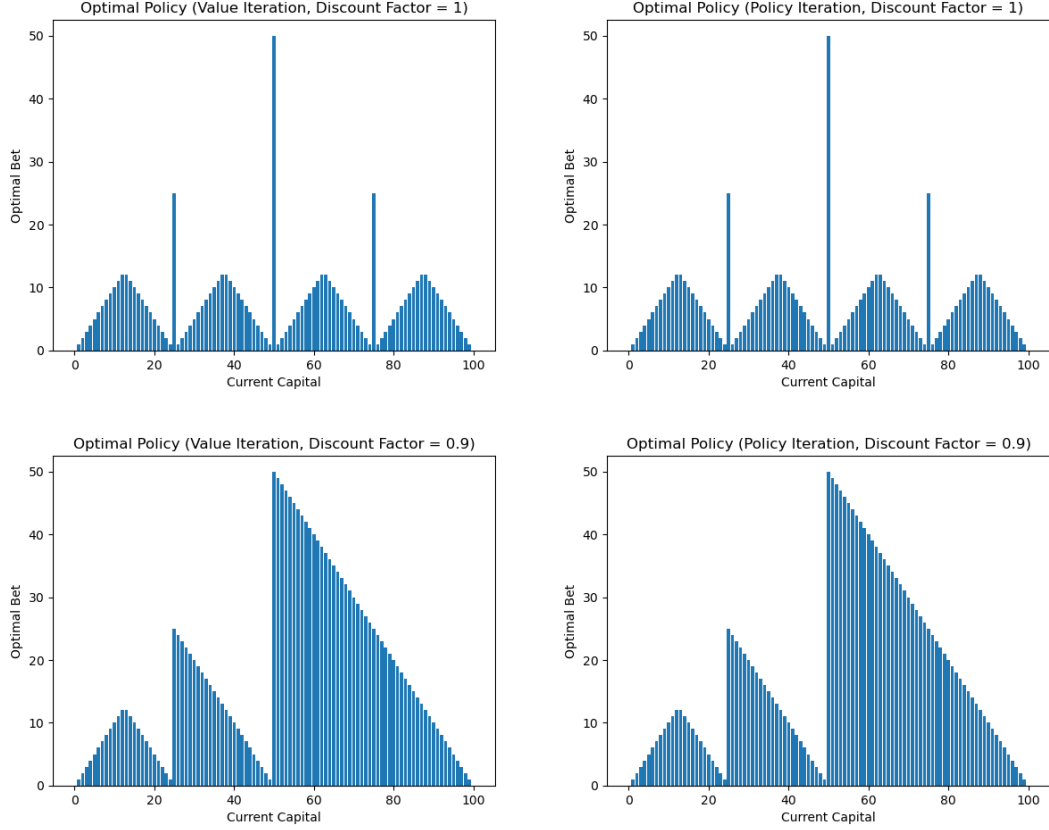


Figure 2: Optimal Policy for Gambler's Problem

4.2 Grid Problem

Map	Algorithm	$\gamma = 0.9$		$\gamma = 0.99$	
		Time (s)	Iters	Time (s)	Iters
4×4	Value Iteration	0.0368	32	0.0509	41
	Policy Iteration	0.0885	5	0.2560	5
7×7	Value Iteration	0.3183	30	0.5971	53
	Policy Iteration	0.6880	6	3.4641	8

Table 2: Time and iteration counts grouped by map size and discount factor γ .

We compare Value iteration (VI) and Policy iteration (PI) on two grid world MDPs: 4×4 (15 states) and 7×7 (40 states) grid. For all grid examples, both policies successfully converged to optimal navigation policies. Figure 3 shows the optimal policy that successfully navigates around obstacles to reach the goal state. The top two are with $\gamma = 0.9$ and $\gamma = 0.99$ respectively, so are the bottom two.

Firstly, for same grid and same discount factor, PI uses substantially fewer iterations but can be slower in wall-clock time due to more expensive iterations, while VI is computationally cheaper per iteration and scales more favorably on the larger mazes. This echoes with the result of Gambler's Problem.

Secondly, the value function (Table 2) shows higher values near the goal state, with decreasing values further away, reflecting the discount factor $\gamma \neq 1$.

Thirdly, for the same grid and the same algorithm (e.g., value iteration), increasing the discount factor γ makes the agent place more weight on long-term rewards, so the propagated utilities tend to become larger and more far-sighted across the map (Figure 3).

In the 4×4 GridWorld, raising γ from 0.9 to 0.99 mainly increases the displayed state values while the overall arrow pattern remains visually unchanged, suggesting the optimal directions are stable under both discount factors.

However, in the 7×7 GridWorld, the optimal policy does change with γ : in the bottom-left corridor region, several arrows that point downward under $\gamma = 0.9$ flip to point upward under $\gamma = 0.99$, indicating that the agent becomes more willing to take actions that pay off over a longer horizon when future rewards are discounted less. Conversely, a smaller γ emphasizes immediate step costs and near-term outcomes, which typically reduces state values and can yield a more "shortsighted" routing preference compared with a higher- γ solution.

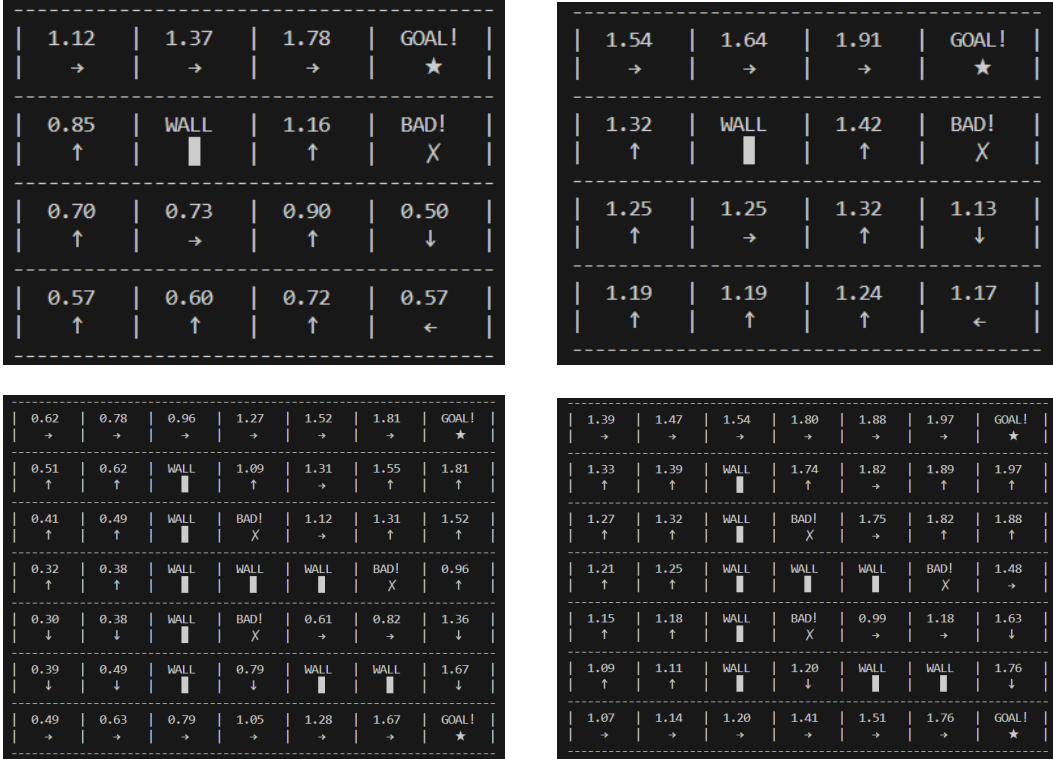


Figure 3: Optimal policy for Gridworld. Arrows show movement direction in each state.

5 Discussion and Conclusion

To conclude this papers, show that MDPs can solve these complex problems with decision making under uncertainty by finding the optimal policy. Specifically, the Gambler's Problem shows how MDPs can model risk-reward tradeoffs in stochastic environments, while Gridworld demonstrates spatial navigation with uncertainty. However different parameters may lead to different solutions, as can be seen when varying γ , which highlights the importance of proper parameter selection for a desired outcome.

And when it comes to finding the optimal policy, Value and Policy iteration will lead to the same solution, but Policy iteration is significantly faster [6], as it performs a complete (or near-complete) policy evaluation and makes significant improvements to the policy directly, essentially skipping the many possible value functions linked to that policy, that Value iteration will explore [6].

References

- [1] Rohan Jagtap. Markov decision process: A beginner's guide to reinforcement learning. <https://builtin.com/machine-learning/markov-decision-process>, 2025. Accessed: 2025-12-20.
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. A Bradford Book, 2 edition, 2018. Accessed: 2025-12-20.
- [3] Denny Britz. Reinforcement learning. <https://github.com/dennybritz/reinforcement-learning>, 2025. GitHub repository; Accessed: 2025-12-20.
- [4] Amy X. Gao. Lecture 18 on markov decision process 1. https://www.cs.toronto.edu/~axgao/cs486686_f21/lecture_notes/Lecture_18_on_Markov_Decision_Process_1.pdf, 2021. Course lecture notes, CS486/686, University of Toronto, Accessed: 2025-12-20.
- [5] Pieter Abbeel. Markov decision processes: Exact methods. <https://people.eecs.berkeley.edu/~pabbeel/cs287-fa12/slides/mdps-exact-methods.pdf>, 2012. Lecture slides for CS 287: Advanced Robotics, University of California, Berkeley, Accessed: 2025-12-20.
- [6] UC Berkeley EECS Department. Mdps: Policies (iterative policy evaluation). <https://inst.eecs.berkeley.edu/~cs188/textbook/mdp/policies-iteration.html>, 2024. Accessed: 2025-12-20.

Github link for source code: https://github.com/dev-stevenp/estr2018_report